Railway Reservation System

Core Java + MySQL Project – Full Description, Database Design & Sample Code

1. Project Overview

The Railway Reservation System is a console-based Java application developed using Core Java and MySQL. The system allows users to view available trains, search trains by source and destination, book tickets, view booking details, check PNR status, and cancel tickets. MySQL is used as the backend database to persist train, passenger, and reservation information. JDBC is used to connect the Java application with the SQL database.

2. Objectives

• Automate basic railway ticket reservation operations.

• Maintain passenger and train information in a structured SQL database.

• Provide booking, cancellation, and PNR-status functionality.

• Demonstrate Core Java, OOP, JDBC, SQL queries, exception handling, and collections.

• Reduce manual data entry and improve consistency of reservation records.

3. Technologies Used

Technology	Purpose
Core Java	Application logic and object-oriented programming
JDBC	Connection between Java application and MySQL
MySQL	Persistent storage for trains, passengers and bookings
SQL	Create, insert, search, update and delete database records
Eclipse / IntelliJ IDEA	Development environment

4. Main Modules

• **Admin/Train Management:** Add, update, delete and display train details.

• **Train Search:** Search trains using source and destination.

• **Passenger Registration:** Capture passenger name, age, gender and contact details.

• **Ticket Booking:** Check seat availability, create a reservation and generate a PNR number.

• **PNR Status:** Retrieve booking details using the PNR number.

• **Ticket Cancellation:** Cancel an existing reservation and update seat availability.

• **Database Management:** Store all persistent records in MySQL using JDBC.

5. Database Design

Table	Important Columns
trains	train_id, train_number, train_name, source, destination, total_seats, available_seats
passengers	passenger_id, name, age, gender, phone
reservations	pnr, train_id, passenger_id, journey_date, seat_no, status, booking_time

6. MySQL Database Script

```
CREATE DATABASE railway_db;
USE railway_db;

CREATE TABLE trains (
```

```
    train_id INT PRIMARY KEY AUTO_INCREMENT,
    train_number VARCHAR(20) UNIQUE NOT NULL,
    train_name VARCHAR(100) NOT NULL,
    source VARCHAR(50) NOT NULL,
    destination VARCHAR(50) NOT NULL,
    total_seats INT NOT NULL,
    available_seats INT NOT NULL
);

CREATE TABLE passengers (
    passenger_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    age INT NOT NULL,
    gender VARCHAR(10),
    phone VARCHAR(15)
);

CREATE TABLE reservations (
    pnr BIGINT PRIMARY KEY,
    train_id INT NOT NULL,
    passenger_id INT NOT NULL,
    journey_date DATE NOT NULL,
    seat_no INT NOT NULL,
    status VARCHAR(20) DEFAULT 'CONFIRMED',
    booking_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (train_id) REFERENCES trains(train_id),
    FOREIGN KEY (passenger_id) REFERENCES passengers(passenger_id)
);

INSERT INTO trains
(train_number, train_name, source, destination, total_seats, available_seats)
VALUES
('12701', 'Godavari Express', 'Hyderabad', 'Chennai', 100, 100),
('12760', 'Charminar Express', 'Hyderabad', 'Chennai', 100, 100);
```

7. Java Database Connection

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {
    private static final String URL =
        "jdbc:mysql://localhost:3306/railway_db";
    private static final String USER = "root";
    private static final String PASSWORD = "your_password";

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

8. Train Search Code

```
import java.sql.*;

public class TrainSearch {
    public static void search(String source, String destination) {
        String sql = "SELECT * FROM trains WHERE source=? AND destination=?";

        try (Connection con = DBConnection.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setString(1, source);
            ps.setString(2, destination);

            ResultSet rs = ps.executeQuery();

            while (rs.next()) {
                System.out.println(
                    rs.getString("train_number") + " | " +
                    rs.getString("train_name") + " | " +
                    rs.getInt("available_seats") + " seats"
                );
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

9. Passenger Insert Code

```
String sql = "INSERT INTO passengers(name, age, gender, phone) VALUES(?,?,?,?)";

try (Connection con = DBConnection.getConnection();
     PreparedStatement ps = con.prepareStatement(sql)) {

    ps.setString(1, name);
    ps.setInt(2, age);
    ps.setString(3, gender);
    ps.setString(4, phone);

    ps.executeUpdate();
    System.out.println("Passenger registered successfully.");
}
```

10. Ticket Booking Logic

```
// Simplified booking flow
// 1. Check available seats.
// 2. Insert passenger.
// 3. Generate PNR and seat number.
// 4. Insert reservation.
// 5. Decrease available seats.

con.setAutoCommit(false);

try {
    // Insert passenger and obtain passenger_id.
    // Read train availability using SELECT ... FOR UPDATE.
    // Insert reservation.
    // Update trains SET available_seats = available_seats - 1.
    con.commit();
    System.out.println("Ticket booked successfully.");
```

```
} catch (Exception e) {
    con.rollback();
    System.out.println("Booking failed. Transaction rolled back.");
} finally {
    con.setAutoCommit(true);
}
```

11. PNR Status Query

```
String sql = "SELECT r.pnr, t.train_name, t.source, t.destination, " +
             "p.name, r.journey_date, r.seat_no, r.status " +
             "FROM reservations r " +
             "JOIN trains t ON r.train_id=t.train_id " +
             "JOIN passengers p ON r.passenger_id=p.passenger_id " +
             "WHERE r.pnr=?";

try (PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setLong(1, pnr);
    ResultSet rs = ps.executeQuery();

    if (rs.next()) {
        System.out.println("PNR: " + rs.getLong("pnr"));
        System.out.println("Passenger: " + rs.getString("name"));
        System.out.println("Train: " + rs.getString("train_name"));
        System.out.println("Seat: " + rs.getInt("seat_no"));
        System.out.println("Status: " + rs.getString("status"));
    }
}
```

12. Ticket Cancellation Query

```
UPDATE reservations
SET status='CANCELLED'
WHERE pnr=? AND status='CONFIRMED';

UPDATE trains
SET available_seats = available_seats + 1
WHERE train_id=?;
```

13. Excel / Report Storage

The primary application data should be stored in MySQL. Excel can be used as an additional reporting/export format for project demonstrations. A reservation report can contain PNR, passenger name, train number, source, destination, journey date, seat number and booking status. The Excel file is not recommended as the primary transactional database because MySQL provides better concurrency, constraints and transaction support.

PNR	Passenger	Train	Source	Destination	Date	Seat	Status
100001	Rahul	12701	Hyderabad	Chennai	2026-08-20	12	CONFIRMED
100002	Anil	12760	Hyderabad	Chennai	2026-08-21	25	CONFIRMED

14. Recommended Project Folder Structure

```
RailwayReservation/
■■■ src/
■   ■■■ DBConnection.java
■   ■■■ Train.java
■   ■■■ Passenger.java
■   ■■■ Reservation.java
■   ■■■ TrainSearch.java
■   ■■■ ReservationService.java
■   ■■■ Main.java
■■■ sql/
■   ■■■ railway_db.sql
■■■ reports/
    ■■■ reservation_report.xlsx
```

15. Key Java Concepts Demonstrated

• Classes and objects

• Encapsulation and constructors

• Inheritance and polymorphism where appropriate

• Interfaces and abstraction

• Collections such as ArrayList and HashMap

• Exception handling

• JDBC and PreparedStatement

• SQL joins, INSERT, SELECT, UPDATE and DELETE

• Transactions using commit and rollback

• Input validation and menu-driven programming

16. Resume Description

Developed a Railway Reservation System using Core Java, JDBC and MySQL to manage train schedules, passenger records, ticket booking, cancellation and PNR status. Implemented OOP concepts, SQL queries, database connectivity, exception handling and transaction management, with reservation data maintained in a relational database and exportable for Excel reporting.